



EJERCICIO PRÁCTICO DE AUDITORÍA DE SOFTWARE

Sistema de Matrículas – Análisis y Automatización con Python

1. Situación planteada

Una institución educativa utiliza un pequeño sistema desarrollado en Python para registrar estudiantes. El sistema permite registrar estudiantes, mostrar información, eliminar registros, calcular una condición relacionada con la edad y guardar los datos en un archivo de texto.

Ustedes asumirán el rol de auditores de software. El objetivo NO es simplemente encontrar errores de programación, sino identificar situaciones que puedan representar riesgos y justificar cada hallazgo mediante evidencia.

2. Archivos de trabajo Valor (1,5)

- sistema_matriculas.py → aplicación que será auditada.
- auditor_matriculas.py → archivo que deberán desarrollar para automatizar parte de la auditoría.

El archivo sistema_matriculas.py será suministrado por el docente. No deben modificarlo inicialmente, porque primero deberán realizar la auditoría sobre el código original.

3. Primera fase: auditoría manual

Antes de programar el auditor, deberán leer y analizar completamente sistema_matriculas.py. La primera parte de la actividad consiste en identificar posibles hallazgos de forma manual.

4. Criterios que deben revisar

Validación de datos: Verificar si los datos ingresados por el usuario son comprobados antes de utilizarse. Pregúntese qué ocurre si un campo queda vacío o contiene información no válida.

Manejo de errores: Verificar si el sistema contempla situaciones inesperadas, especialmente cuando convierte datos o trabaja con archivos.

Seguridad de la información: Revisar cómo se maneja y almacena la información personal de los estudiantes.

Gestión de archivos: Analizar qué ocurre si el archivo no puede abrirse, escribirse o utilizarse correctamente.

Integridad de los datos: Verificar si pueden generarse registros duplicados o eliminarse datos de manera incorrecta.

Mantenibilidad: Analizar si la estructura del programa facilita realizar cambios y mantener el sistema en el futuro.



5. Registro de hallazgos Valor (2)

Por cada posible hallazgo encontrado deberán registrar como mínimo:

1. ID del hallazgo.
2. Categoría.
3. Número de línea o ubicación aproximada.
4. Evidencia encontrada en el código.
5. Descripción clara del hallazgo.
6. Riesgo o consecuencia posible.
7. Nivel de severidad: Alta, Media o Baja.
8. Recomendación de mejora.

La matriz de auditoría deberá tener una estructura similar a la siguiente:

ID	Categoría	Línea	Evidencia	Hallazgo	Riesgo	Severidad	Recomendación

6. Preguntas orientadoras Valor (1,5)

Responda las siguientes preguntas de la auditoría:

- ¿Qué ocurre si el usuario deja vacío uno de los campos?
- ¿Qué ocurre si el usuario introduce texto donde el sistema espera un número?
- ¿Qué información personal se almacena?
- ¿Cómo se almacena esa información?
- ¿Qué sucede si el archivo de datos no está disponible?
- ¿Puede registrarse dos veces el mismo documento?
- ¿Qué ocurre si se intenta eliminar un estudiante que no existe?
- ¿La lógica de eliminación garantiza que se afecte únicamente el registro esperado?
- ¿Las funciones tienen responsabilidades claramente separadas?
- ¿Qué partes del programa podrían ser difíciles de modificar posteriormente?

7. Segunda fase: desarrollar el auditor en Python

Después de realizar la auditoría manual, deberán construir auditor_matriculas.py. La herramienta deberá leer el archivo sistema_matriculas.py y automatizar al menos una parte de los criterios revisados.

8. Entregables

- Matriz de auditoría con los hallazgos identificados manualmente.
- Archivo auditor_matriculas.py.
- Captura de pantalla de la ejecución del auditor.
- Conclusión de la auditoría.
- Breve explicación de las diferencias entre los hallazgos manuales y los detectados automáticamente.



9. Conclusión que deberán responder

Al finalizar, respondan en un párrafo:

¿Qué ventajas y limitaciones tiene utilizar una herramienta desarrollada en Python para apoyar una auditoría de software?

10. Reglas de la actividad

- No modificar el sistema auditado antes de terminar la primera fase.
- Todo hallazgo debe estar respaldado por evidencia.
- No afirmar que algo es un problema sin explicar el riesgo o la razón.
- Diferenciar entre un posible hallazgo y una vulnerabilidad confirmada.
- La herramienta automática debe apoyar el análisis, no reemplazar el criterio del auditor.
- El código desarrollado por el estudiante debe estar organizado y ser comprensible.